# Reproducible Statistical Workflows

## Version Control + Daily Practice from Data Wrangling to Analysis

StatHarbor Analytics

# Who this course is for

## Audience

Daily statistical programmers, analysts, and data scientists who:

- Build pipelines (R/Python), run models, produce tables/figures
- Need results to be reproducible for review, collaboration, or audit
- Feel overwhelmed by Git and want a minimal workflow that works

## Outcome

You will leave with a concrete daily routine and a project structure that prevents: data confusion, irreproducible results, and "final_final_v7" chaos.

# The real problems we face daily

## This is what actually breaks projects

- A dataset is refreshed (row count changes) and results drift
- A colleague reruns your notebook and gets different estimates
- The censoring definition changes and Table 3 no longer matches
- You tried 5 models and cannot remember which one produced the figure
- Someone asks: "Which version produced this output?" and you cannot answer

### Core message

Version control is not developer culture. It is statistical risk management.

# Course map

## Part I: Data Wrangling (Invisible Risk Zone)

Project structure, data versioning, deterministic cleaning, feature engineering logs, missing data decisions, and how to commit wrangling like a professional.

## Part II: Analysis & Modeling (Accountability Zone)

Estimands, model tracking, survival/PS changes, ML experiment control, leakage prevention, and reproducible reporting (tables/figures) with an auditable trail.

### Hands-on

You will see "lab slides" that tell you exactly what to do in a real project.

# Minimum Git you need (today)

## The daily 6

- `git status` (what changed?)
- `git diff` (what changed exactly?)
- `git add <file>` (stage carefully)
- `git commit -m "..."` (checkpoint)
- `git pull` (sync before you start)
- `git push` (backup + share)

## Rule

If results would change, commit.

# The mental model: snapshots, not magic

### Repository

A folder whose history is tracked.

### Commit

A snapshot + a message explaining **why** it exists.

### Stat rule of thumb

Commit when you would otherwise create `analysis_final_v7.R`.

# Commit messages for statistical work

## Pattern that scales

`type: short intent` (type = clean/feat/fix/tune/doc/data/report)

### Good messages

- `data: update extract v2026_02_15 (rowcount +1.2%)`
- `clean: recode missing BMI from 0 to NA`
- `feat: add time-varying covariate alignment`
- `fix: correct censoring definition (death vs loss-to-followup)`
- `tune: RF trees 500 -> 1000, max_depth 5 -> 10`
- `report: regenerate Table 1 + KM figure with new spec`

# Part I — Data Wrangling

### The invisible risk zone

Most real-world errors and bias are introduced **before** modeling. This part teaches how to build deterministic, auditable pipelines.

# Project structure that prevents chaos

## Recommended structure

```
project/
  README.md
  data_raw/ # never edited, never committed (usually)
  data_processed/ # derived datasets (optional to commit)
  src/
    01_import/
    02_clean/
    03_features/
  analysis/
    01_eda/
    02_models/
  outputs/
    figures/
    tables/
  reports/
  renv.lock OR requirements.txt
```

# The Golden Rule: never commit sensitive raw data

**Never commit patient-level raw data**

Git is for code and small text artifacts. Even private repos can leak via logs, forks, or misconfiguration.

## What you *do* commit

- Data extraction scripts (SQL, API calls, pull logic)
- Data dictionary / schema docs
- Metadata: row counts, date ranges, version tags
- Checksums/hashes (when allowed)

# `.gitignore` for statistical projects

## Common ignores

- Data: `*.csv`, `*.rds`, `*.parquet`, `*.sas7bdat`
- Secrets: `.env`, `*.key`, `*.pem`
- Cache: `__pycache__/`, `.Rhistory`, `.ipynb_checkpoints/`
- Outputs (optional): `outputs/` if regenerated

## Practical rule

Commit outputs only when you need an auditable snapshot (e.g., submission package).

# Data versioning (without committing data)

## What to record every time data changes

- Extract date / snapshot label
- Row count + key counts (patients, visits, events)
- Column count + schema version
- Filters applied (inclusion criteria)

### Commit trigger

```
data:  update extract v2026_02_15 (n=142,018; events=9,402)
```

# Lab 1 — Create a data receipt (5 minutes)

## Deliverable

A small text file: `data_receipt.md` saved in the repo.

## Template (copy/paste)

- Data snapshot: ____
- Extract date: ____
- Rows: ____ Unique IDs: ____
- Time window: ____
- Inclusion filters: ____
- Notes: ____

## Commit

```
doc: add data receipt for snapshot v2026_02_15
```

# Deterministic wrangling: the non-negotiable standard

**Wrangling must be deterministic**

Same inputs $\Rightarrow$ same outputs. Always.

## Common sources of non-determinism

- Unsorted joins (many-to-many surprises)
- Random sampling without fixed seed
- Implicit ordering (depends on file read order)
- Floating parsing differences (date/time/timezone)

# Join logic: where silent errors happen

## Most silent bugs are join bugs

- Duplicate keys create row explosions
- Many-to-many joins change weights implicitly
- Left join vs inner join changes the population

### Practical habit

Before join: check key uniqueness. After join: check row count and key count.

# Lab 2 — Join safety checklist

## Before the join

- Are keys unique in each table?
- If not, is this intended (visits, repeated measures)?
- What should happen to unmatched rows?

## After the join

- Did row count change as expected?
- Did unique patient count change?
- Did event count change?

## Commit if population changes

```
clean:  switch to inner join on id+index_date (restrict cohort)
```

# Missing data: decisions must be logged

**Missingness decisions change estimands**

Dropping rows, imputing, or recoding missing values changes the target population.

## Commit messages

- `clean: treat BMI==0 as missing`
- `feat: add missingness indicators for labs`
- `analysis: switch to multiple imputation (m=20)`

# Feature engineering: where bias sneaks in

Feature engineering can change conclusions

- Winsorization / trimming
- Categorizing continuous covariates
- Creating composite scores
- Standardization / centering
- Time alignment (baseline vs time-varying)

**Practical rule**

If a feature definition changes, commit it as a `feat:` or `fix:`.

# Wrangling outputs: what to save

## What to write out

- Processed dataset *(optional, depending on policy)*
- QA summary tables (counts, missingness, duplicates)
- Data dictionary (final variables used in modeling)

### Recommended

Always save a QA report as text/markdown. It is lightweight and auditable.

# Part I recap: Wrangling checklist

Before you model, confirm

- Data receipt exists (snapshot, rows, IDs, window)
- Joins are validated (keys, row counts)
- Missingness decisions logged
- Feature engineering definitions stable
- Pipeline deterministic (seed, sorting, time)

If any of these change, commit.

# Part II — Analysis & Modeling

### The accountability zone

This is where you must be able to answer: "Which version of code + data produced this result?"

# Start with the estimand (not the model)

## Before you run anything

Define:

- Population, treatment/exposure, outcome
- Time horizon and risk window
- Censoring definition
- Estimand (ATE/ATT, HR, RMST, etc.)

### Commit

```
analysis:  define ATT estimand, 2y horizon, censor LTFU only
```

# Model tracking like a scientist

## Replace file chaos with commit history

You do not need a new filename for every model. Use Git as your experiment log.

### Examples

- `feat: add Cox model with baseline covariates`
- `tune: add spline for age (df=4)`
- `fix: remove post-treatment covariate from PS model`
- `analysis: switch to robust SEs (cluster=id)`

# Survival analysis: commit the definitions

## Survival models are definition-sensitive

Small changes can produce large differences:

- Time origin
- Censoring rules
- Competing risks handling
- Landmarking / delayed entry

## Commit examples

- `fix: set time origin to index_date (not enrollment)`
- `analysis: treat death as competing risk (Fine-Gray)`

# Propensity scores: what to track

## PS work is mostly specification changes

- Covariate set (baseline only vs time-varying)
- Functional form (linear vs splines/interactions)
- Trimming / overlap restrictions
- Balance diagnostics thresholds

### Commit examples

- `feat: add overlap trimming (0.01, 0.99)`
- `tune: add interaction age*sex to PS`
- `report: add love plot + SMD table`

# ML pipelines: prevent leakage or your CV is fake

### Leakage pattern

Preprocess entire dataset, then split $\Rightarrow$ performance is inflated.

### Practical standard

All preprocessing must be fit on training folds only:

- scaling
- imputation
- encoding
- feature selection

### Commit

```
fix:  move preprocessing inside sklearn Pipeline
```

# Lab 3 — The “result drift” investigation

## Scenario

Yesterday your estimate was 0.82. Today it is 0.91. Why?

## Steps

- `git diff` (did code change?)
- Check data receipt (did data change?)
- Check seed (did RNG change?)
- Check dependency lock (did packages change?)

## Rule

Never trust a changed result until you identify the reason.

# Notebooks: how to use them without chaos

## Use notebooks for exploration

Keep stable pipelines in scripts:

- `src/` for functions
- `analysis/` for reproducible runs
- notebook only for narrative + plots

## Practical steps

- Clear outputs before commit
- Keep one "run-all" cell that calls scripts
- Consider syncing notebooks to text via `jupytext`

# Undo safely (auditable)

### Uncommitted mistake

`git restore <file>` (back to last commit)

### Committed mistake

`git revert <commit>` (new commit that undoes)

**Why revert is preferred in serious work**

It preserves history, which matters for review and audit.

# Branching for experiments

## Stable main, experimental branches

`main` = what you can defend. Branches = what you test.

```
git checkout -b feature/ps-trimming
# edit, run, validate...
git add src/analysis_ps.R
git commit -m "feat: add PS trimming (0.01, 0.99)"
git checkout main
git merge feature/ps-trimming
```

### Rule

If an experiment fails, delete the branch. Your main stays clean.

# Reproducibility stack (the full picture)

Reproducibility =

- Code (Git commit)
- Data snapshot (receipt + metadata)
- Environment (lockfile)
- Random seed
- Execution entry point (one script to rebuild outputs)

**Practical goal**

A new person should be able to rebuild outputs by running one command/script.

# Lab 4 — Make a one-click rebuild

## Deliverable

A script that rebuilds outputs:

- R: `Rscript analysis/run_all.R`
- Python: `python analysis/run_all.py`

## What it does

- Loads processed data
- Runs models
- Saves tables/figures to `outputs/`

## Commit

```
feat: add one-click rebuild pipeline (run_all)
```

# Reporting: tables and figures as products

## Outputs are products

Tables/figures must be tied to:

- Commit hash
- Data snapshot
- Model spec

### Practical tip

Write the commit hash into the table/figure caption or a footer.

# Industry standard: answering the audit question

The question that matters

"Which exact version produced Table 3?"

**Your answer should include**

- Git commit (hash)
- Data snapshot label (receipt)
- Environment lockfile
- Run script name

This is what "control" looks like.

# Daily routine (StatHarbor)

## Morning (5 minutes)

- `git pull`
- Verify data snapshot label
- Confirm tasks for the day

## During work

- Commit when results could change (definitions/specs/features)
- Keep wrangling and analysis separate
- Use branches for risky experiments

## End of day (5 minutes)

- `git status` clean up
- Commit checkpoint
- `git push`

# Course wrap-up: your minimum viable professional standard

## If you can do these 6 things

- Keep raw data out of Git
- Maintain a data receipt
- Commit when results could change
- Use branches for experiments
- Lock your environment
- Provide a one-click rebuild script

You are already in the top tier of reproducible practice.

# StatHarbor Takeaway

**Git is not about coding.**

It is about statistical integrity.

**Version control is part of scientific thinking.**

If your work cannot be reproduced and audited, it is not under control.